

HDFS has a master/slave architecture, wherein a cluster has a single component called NameNode, which acts as the master server. NameNode manages the file system namespace (files, directories and blocks, as well as their relationships). The namespace is stored in the memory and changes are persisted to the disk on a periodic basis.

In addition to that, there are slave components called DataNodes, usually one per node in the cluster. These processes manage the storage attached to the specific compute node they run on.

NoSQL databases (<http://nosql-database.org/>): As the Web has grown and become more accessible, it has become apparent that the existing relational database technologies are not equipped to handle the huge volumes and concurrency requirements of Web 2.0. To meet this need 'Not only SQL' databases have emerged as alternate data storage and management systems. While HDFS and Hadoop are data processing engines for analytics use cases, either in batch or real-time, NoSQL databases essentially serve as data storage layers for Web based front-end systems that need large concurrent data handling capacities.

Some of the key features that characterise these databases are that they are usually schema-free (or have a minimal schema structure), horizontally scalable, and rely on eventual consistency models rather than immediate consistency models.

There are four basic architectures of NoSQL databases that have emerged. These are:

- Key/Value stores are based on the data model of hash data structures or associative arrays. These systems were based on Amazon's DynamoDB paper (<http://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>).
- Columnar databases (DBs) are based on Google's BigTable paper (<http://research.google.com/archive/bigtable-osdi06.pdf>). The data model here is that each row can have its own schema, e.g., HBase and Cassandra.
- Document DBs are systems where the data is stored as documents, and each document is a collection of Key/Value pairs. Generally, these are expressed as JSON documents (e.g., MongoDB and CouchDB).
- Graph DBs are systems where the data models are represented as nodes, or as relationships between the nodes. Both nodes and relationships are represented in key-value pairs (e.g., Neo4J).

Tachyon (<http://tachyon-project.org/>): This is a platform that provides reliable in-memory data sharing across cluster frameworks and jobs. Tachyon essentially sits on top of storage platforms such as HDFS, thereby providing memory-centric data processing capabilities across cluster frameworks and jobs. Although some of the existing cluster computing frameworks like Spark, etc, have leveraged in-memory data

processing, there were three key shortcomings that motivated the development of Tachyon:

- Although jobs processed data within memory, sharing of data across jobs and frameworks was not achieved as the data was only available within the JVM context of the job.
- As the execution engine and storage was within the same JVM context, any execution engine crash led to the loss of data and necessitated re-computation.
- Data was replicated in-memory across jobs in certain cases leading to a larger data footprint, and heavier garbage collection.

Tachyon was developed to solve the above problems and was driven by a need to push lineage down to the storage layer. It enables storing of only one copy of data in the memory, which is made available across all frameworks such as Spark, MapReduce, etc. Moreover, fault tolerance was enabled by leveraging re-computations using lineage.

Data processing frameworks

Once the data is persisted into a storage layer, the next step is to process this data to derive insights. There are several frameworks that we will look at here.

The Apache Hadoop stack (<https://hadoop.apache.org/>) is the grand-daddy of Big Data processing frameworks, and has become the de-facto platform into which the technology has largely converged. The cost-effectiveness and the scalability of the platform is a perfect match for the needs of the large scale data processing in the industry. In addition, the reliability and the community support around the platform and the ecosystem have led to a wider adoption of the platform.

The Hadoop ecosystem has three main goals:

Scalability – enabling scaling to cater to larger requirements with just the addition of nodes to the cluster. This is further amplified by the fact that the framework relies on the local computation model to derive benefits from the simplified scalability model.

Flexibility – to provide the flexibility to store differently structured data formats. This is enabled by the 'Schema on Read' approach, which enables the system to store anything, and only decipher the schema at the time of reading the data, which is when it is required to know the data.

Efficiency – to ensure the cluster resources are optimally utilised for higher efficiency.

Hadoop MapReduce (<https://hadoop.apache.org/>) is the implementation of the MapReduce programming paradigm (popularised by a Google paper). This programming paradigm is modelled to process very large datasets in parallel, on large clusters, while ensuring reliability and fault-tolerance.

The MapReduce() paradigm itself is founded on the concept of a distributed file system which ensures reliability and scalability. A MapReduce() program consists of two procedures – Map() and Reduce(). The Map() procedure processes the input dataset in parallel and emits a processed